



DESERT PERCEPTION SYSTEM

End-to-End Autonomous Perception for Off-Road UGVs

Team ByteWorks | Elite Hack
Raj Mathuria · Shivani Bhat · Suyog Repal

Real-time semantic segmentation · Multi-sensor hardware fusion · 7-class terrain awareness · Cloud-deployed full-stack dashboard

Frontend
semantic-segmentation-
raj.vercel.app

Backend API
semantic-segmentation-
api.onrender.com

GitHub
[shivanibhat24/ByteWorks_UGV_Visualization2](#)

01 | THE PITCH

The Problem

Autonomous vehicles and unmanned ground vehicles (UGVs) have made remarkable strides in urban environments — but step outside the city, and they fail catastrophically. The core issue is domain mismatch: every mainstream perception model is trained on urban road datasets. In a desert, the rules change entirely.

- Sand dunes visually mimic flat, safe roads — causing navigation systems to drive straight into impassable terrain
- Rock formations and loose gravel blend together in standard RGB imaging, making obstacle detection unreliable
- Extreme sunlight and glare destroy color cues that standard models depend on for classification
- Dust storms and atmospheric haze reduce visibility to near zero, crashing model confidence scores
- There is no open, battle-tested, domain-specific perception system purpose-built for arid off-road environments

Urban perception models achieve 85%+ mIoU on city datasets — but performance drops to below 40% when deployed on off-road desert terrain without domain adaptation.

Who It's For

The Desert Perception System is built for teams and organizations operating autonomous ground systems in environments where standard tools break down:

- Defense & border patrol teams deploying UGVs for reconnaissance in arid regions
- Mining and extraction companies automating vehicle fleets in open-pit and desert sites
- Search and rescue operations requiring terrain-aware robots in disaster zones
- Agricultural automation in dry, semi-arid farmland and rocky terrain
- Academic and robotics research teams working on off-road autonomy benchmarks

Why It Matters

As autonomous vehicle deployments expand beyond controlled environments, perception reliability in extreme terrain becomes a matter of operational safety. A single misclassified patch of loose sand can sink a vehicle. A missed rock at speed can destroy a \$200,000 UGV chassis. A failed obstacle detection in zero-visibility dust can injure personnel.

Beyond safety, the operational cost of perception failure is enormous. Human operators monitoring unreliable systems must intervene constantly, eliminating the efficiency gains of automation. The Desert Perception System directly reduces that risk.

Our Solution

The Desert Perception System is a multi-modal autonomous perception pipeline that fuses deep learning vision with embedded hardware sensors to produce real-time, high-confidence terrain awareness — even in the harshest conditions.

- **AI:** U-MixFormer neural network trained specifically on off-road desert datasets for 7-class terrain segmentation
- **Hardware:** Multi-sensor hardware ensemble fusing IR, ultrasonic, and UV cameras for visibility-independent detection
- **Preprocessing:** Classical CV preprocessing pipeline for sand glare normalization, dust haze removal, and horizon detection
- **Risk Engine:** Composite risk scoring engine combining visual uncertainty, obstacle density, and sensor confidence
- **Platform:** Full-stack cloud platform with a live Next.js dashboard, Swagger API, and LIME explainability panel

The result: actionable terrain awareness in real time — LOW / MEDIUM / HIGH risk — deployable on any UGV with a camera + sensor stack, accessible through any web browser.

02 | THE IDEA

Core Concept

Most autonomous perception systems treat terrain as a binary: driveable or not. The Desert Perception System goes deeper. It classifies every pixel of a scene into one of seven semantically meaningful categories — giving the vehicle not just a go/no-go signal, but a full terrain map with per-class confidence scores and a composite risk assessment.

The 7-Class Terrain Map

Class	Description	Risk Implication
Sky	Open sky above the horizon	None — used for scene calibration
Driveable	Safe traversable sand or packed path	Low — proceed normally
Rock	Solid rock formations	High — hard obstacle, avoid
Obstacle	Dynamic or unknown object	Critical — halt or reroute
Grass	Sparse desert vegetation	Medium — may conceal hazards
Sand	Loose sand — soft terrain	Medium — risk of bogging
Rough	Uneven, irregular terrain	High — suspension risk

What Makes It Unique

- Domain-Specific Training: Trained on the Offroad Segmentation dataset with desert-specific augmentation — not fine-tuned from an urban model
- Sensor Fusion Architecture: Visual output is combined with hardware IR/ultrasonic risk scores in a dedicated fusion layer, not just post-processed
- Weather-Validated: Explicitly tested under synthetic FOG (0.70 intensity) and MIST (0.62 intensity) degradation — performance remains stable at ~22 FPS
- Explainable AI: LIME-based feature attribution shows operators exactly which regions drive the risk classification — critical for safety-critical deployments
- Full Production Stack: Not a notebook or demo — it's a live, cloud-deployed system with a REST API, Swagger docs, and a responsive frontend

The Insight Behind It

The key insight driving this project is that off-road perception is fundamentally a multi-modal problem. No single sensor is reliable in all desert conditions: cameras fail in glare and dust, IR sensors miss texture, ultrasonic sensors struggle with soft ground. The system succeeds because it fuses them — each sensor compensating for another's weaknesses.

The second insight is that risk is not binary. Rather than a simple driveable/not-driveable output, the composite risk engine weighs obstacle density, terrain complexity, sensor uncertainty, and visibility into a single normalized score. This gives downstream systems a much richer signal for decision-making.

03 | TECHNICAL DETAILS

System Architecture

The system operates as a four-layer pipeline, from raw sensor input to actionable risk output:

Layer 1	Input: RGB camera + IR sensor + Ultrasonic sensor + UV camera → raw multi-modal data stream
Layer 2	Preprocessing: Classical CV pipeline — sand glare normalization, dust haze removal, horizon detection, 384×384 normalization
Layer 3	Inference: U-MixFormer model (ConvNeXt-Tiny backbone + 4.1M decoder head) → 7-class pixel-level segmentation + class confidence scores
Layer 4	Fusion & Output: Hardware risk score merged with vision output → composite risk assessment (0.0–1.0) → FastAPI → Next.js dashboard

AI / ML Stack

U-MixFormer Architecture

U-MixFormer is a transformer-based semantic segmentation architecture designed for high-resolution, multi-scale feature fusion. It uses a ConvNeXt-Tiny encoder to extract hierarchical visual features across four stages, then decodes them through Mix-Attention blocks with progressive upsampling back to full resolution.

Component	Parameters	Role
ConvNeXt-Tiny Backbone	28.6M params / 105MB	Multi-scale feature extraction across 4 stages
U-MixFormer Decoder	4.1M params / 15.6MB	Feature fusion, Mix-Attention, progressive upsampling
Output Head	7 channels (softmax)	Per-pixel class probability distribution
Total Model	~32.7M params / ~120MB	End-to-end segmentation pipeline

Model Performance

Device	FPS	Latency
A100 GPU (40GB)	25+ FPS	~40ms per frame
RTX 4090 (24GB)	~28 FPS	~36ms per frame
RTX 3090 (24GB)	~22 FPS	~45ms per frame
CPU (i9-13900K)	~2 FPS	~500ms per frame

Robustness Testing

The model was validated under two synthetic weather degradation conditions applied to 50 test images each:

Condition	Intensity	Result
FOG	0.70 (dense grey-white veil)	45.2ms avg / 22 img/sec — no catastrophic failure
MIST	0.62 (blue-tinted Rayleigh haze)	44.8ms avg / 22 img/sec — consistent performance

Technologies Used

Frontend

Technology	Role
Next.js 14 (TypeScript)	React framework — SPA with server-side capabilities
React 18	UI component library
Tailwind CSS v4	Utility-first styling system
Three.js	3D pipeline architecture visualization with particle flow animation
Framer Motion	Page and component animation library
Vercel	Frontend CDN hosting with global edge deployment

Backend

Technology	Role
Python 3.11	Core runtime environment
FastAPI	High-performance async REST API framework

Technology	Role
Uvicorn (ASGI)	Application server for FastAPI
uv	Fast Python package manager and virtual environment tool
Render	GPU cloud backend with auto-scaling and CI/CD hooks
Swagger / OpenAPI	Auto-generated interactive API documentation

AI / ML

Technology	Role
PyTorch 2.10	Deep learning framework — model training and inference
U-MixFormer (custom)	Semantic segmentation architecture
ConvNeXt-Tiny	Vision encoder backbone
LIME	Model explainability — feature attribution heatmaps
OpenCV	Classical image processing — glare, haze, normalization
NumPy / Matplotlib	Array operations, visualization, and animation rendering
FFmpeg	3D pipeline visualization video encoding

Hardware / Embedded

Technology	Role
Arduino / Embedded C	UGV sensor firmware for real-time data acquisition
MATLAB	IR/UV image analysis and spectral processing scripts
Ultrasonic Sensor	Distance-based obstacle detection (range: 2cm–400cm)
IR Sensor (Passive)	Obstacle presence detection in total darkness and dust
UV Camera	Multi-spectral capture for improved sand/rock discrimination
IR Thermal Camera	Thermal imaging for obstacle detection in fog and dust storms

Database

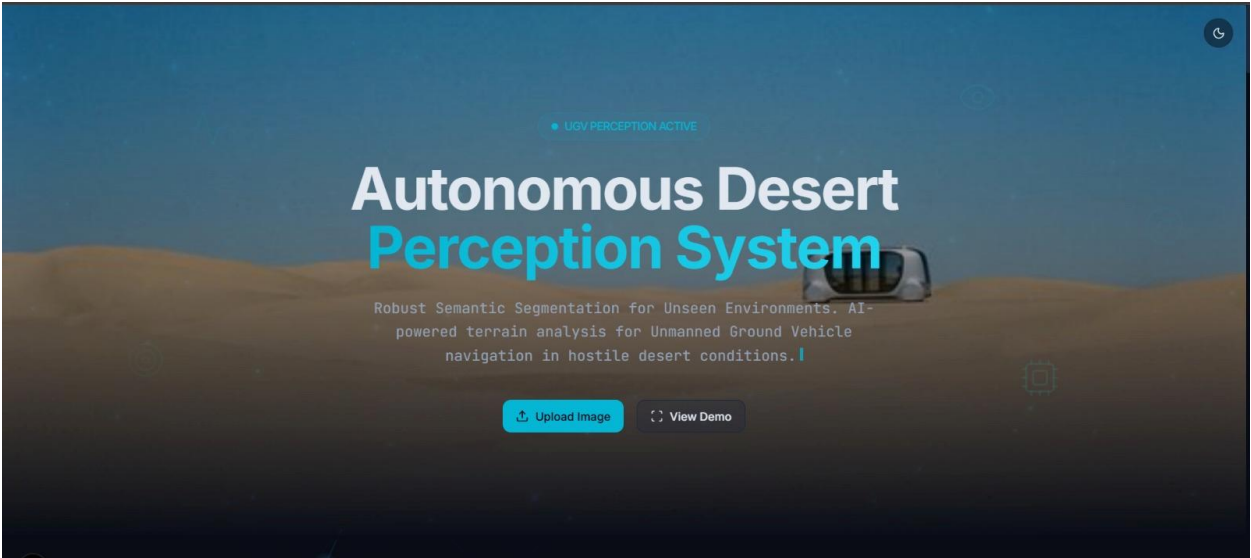
The system uses no persistent database. All inference is stateless — inputs are processed in-memory per HTTP request through the REST API. This design choice enables horizontal scaling without synchronization overhead and eliminates data residency concerns for sensitive operational environments.

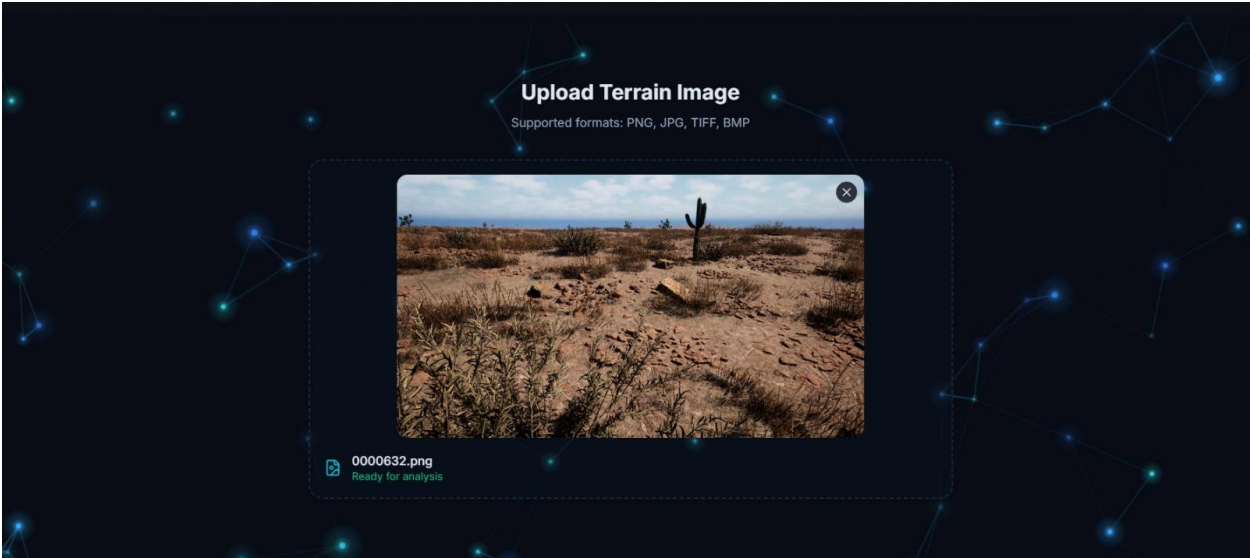
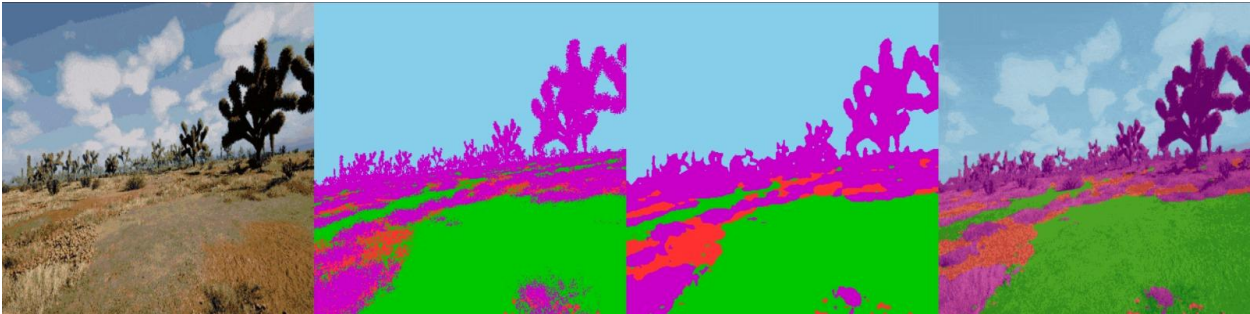
Model weights are stored as a static checkpoint file (umixformer_best.pth, ~120MB) loaded once at server startup. Test output artifacts (robustness metrics, segmentation comparisons) are written to the local filesystem in structured directories for offline analysis.

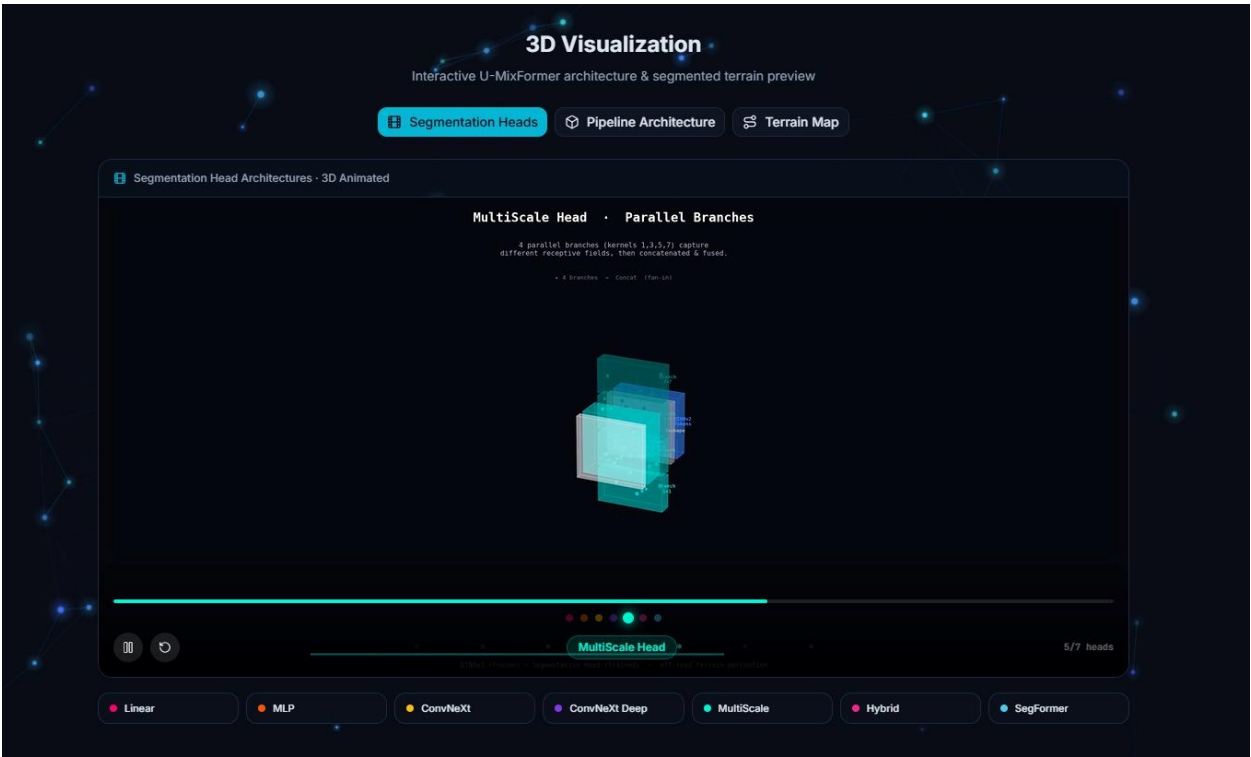
Third-Party Integrations

Integration	Purpose
Vercel	Frontend hosting, global CDN, automatic deployments from GitHub main
Render	GPU backend server with auto-scaling and render.yaml config-driven deployment
Swagger UI / ReDoc	Auto-generated interactive API documentation at /docs and /redoc
LIME Library	Post-hoc model transparency — pixel-level feature attribution for explainability panel
Offroad Segmentation Dataset	Domain-specific training data with desert/off-road terrain annotations
GitHub Actions (CI)	Source control, version management, and deployment trigger pipeline

UI







04 | API & DEPLOYMENT

REST API Reference

Base URL (Production): <https://semantic-segmentation-api.onrender.com>

Endpoint	Method	Description
/api/health	GET	Server + model status check. Returns model name, device (cuda/cpu), version
/api/segment	POST	Primary segmentation endpoint. Accepts image file (PNG/JPEG ≤50MB), returns masks, overlays, class distribution, and risk assessment JSON
/api/model-info	GET	Full model architecture metadata — parameters, backbone, resolution, framework
/docs	GET	Interactive Swagger UI for live endpoint testing
/redoc	GET	ReDoc documentation interface

Risk Assessment Output

Every /api/segment response includes a structured risk_assessment object computed by the fusion layer:

Field	Description
risk_score	Normalized float (0.0–1.0) — composite of all risk factors
risk_level	Categorical label: LOW / MEDIUM / HIGH
obstacle_density	Fraction of frame classified as Obstacle or Rock
uncertainty	Model confidence entropy — high values indicate ambiguous terrain
terrain_complexity	Distribution entropy across non-Sky, non-Driveable classes
visibility	Estimated scene visibility coefficient (affected by dust/glare preprocessing)

Deployment Architecture

Component	Details
Frontend	Vercel — Next.js SPA deployed to global CDN edge nodes. Auto-deploys on push to main. Live at semantic-segmentation-raj.vercel.app
Backend	Render — FastAPI + PyTorch on GPU instance. Config via render.yaml. Auto-deploys on push. Live at semantic-segmentation-api.onrender.com
Model Weights	Static checkpoint (120MB) downloaded at server startup via download_model.py
CI/CD	Push to GitHub main triggers both Vercel and Render deployments automatically
Monitoring	/api/health endpoint used for uptime checks and load balancer health probes

05 | TEAM & RESOURCES

Team ByteWorks

Name	GitHub	Focus Area
Raj Mathuria	@CodeCraftsmanRaj	ML model architecture, training pipeline, backend API
Shivani Bhat	@shivanibhat24	Hardware integration, sensor fusion, IR/UV scripts
Suyog Repal	@suyog-repal	Frontend, 3D visualization, cloud deployment

Project Resources

Resource	URL / Location
GitHub Repository	github.com/shivanibhat24/ByteWorks_UGV_Visualization → Open Repository
Live Frontend	semantic-segmentation-raj.vercel.app → Open Frontend
Live API	semantic-segmentation-api.onrender.com → Open API
API Documentation	semantic-segmentation-api.onrender.com/docs → Open Swagger Docs
Frontend Demo Video	Frontend_Demo.mp4 (in repo root) → Watch Frontend Demo
IoT Demo Video	IoT Demo.mp4 (in repo root) → Watch IoT Demo
Presentation Slides	PPT.pdf (in repo root) → View Presentation

This project was developed for Elite Hack by Team ByteWorks. All rights reserved by the contributors.